

Patrones de cada Sprint y su ubicación en cada Sprint

Sprint1: Prefijo Base:

PracticasEmpresariales/backend/src/main/java/co/edu/cue/practicas/

SINGLETON

- config/singleton/SystemConfig.java
- audit/singleton/AuditoriaLogger.java
- config/SecurityConfig.java
- security/jwt/JwtUtil.java

PROXY

- security/filter/ScopeValidationAspect.java
- security/jwt/JwtAuthenticationFilter.java
- security/annotation/RequiereRol.java
- security/annotation/SoloLectura.java

OBSERVER

- event/AuditoriaEvent.java
- event/UsuarioCreadoEvent.java
- event/listener/AuditoriaEventListener.java
- event/listener/NotificacionEventListener.java

BUILDER

- service/programa/ProgramaBuilder.java

MEDIATOR

- service/dashboard/DashboardMediator.java
- service/dashboard/DashboardService.java
- controller/dashboard/DashboardController.java

Sprint 2:

PROTOTYPE — GPE-141 / GPE-145

- `src/main/java/co/edu/cue/practicas/pattern/prototype/IPrototype.java`
- `src/main/java/co/edu/cue/practicas/pattern/prototype/CatalogoPracticaPlantilla.java`
- `src/main/java/co/edu/cue/practicas/pattern/prototype/EmpresaPlantilla.java`

BUILDER — GPE-141 / GPE-143 / GPE-146 / GPE-150 / GPE-152

- `src/main/java/co/edu/cue/practicas/pattern/builder/CatalogoPracticaBuilder.java`
- `src/main/java/co/edu/cue/practicas/pattern/builder/CatalogoPracticaDirector.java`
- `src/main/java/co/edu/cue/practicas/pattern/builder/EmpresaBuilder.java`
- `src/main/java/co/edu/cue/practicas/pattern/builder/VacanteBuilder.java`
- `src/main/java/co/edu/cue/practicas/pattern/builder/VacanteDirector.java`
- `src/main/java/co/edu/cue/practicas/pattern/builder/ExpedienteBuilder.java`

STRATEGY — GPE-145 / GPE-147

- `src/main/java/co/edu/cue/practicas/pattern/strategy/EstrategiaValidacion.java`
- `src/main/java/co/edu/cue/practicas/pattern/strategy/EstrategiaValidacionEstandar.java`

CHAIN OF RESPONSIBILITY — GPE-145

- `src/main/java/co/edu/cue/practicas/pattern/chain/ValidadorAptitud.java`

- `src/main/java/co/edu/cue/practicas/pattern/chain/ContextoValidacion.java`
- `src/main/java/co/edu/cue/practicas/pattern/chain/ValidadorCatalogoActivo.java`
- `src/main/java/co/edu/cue/practicas/pattern/chain/ValidadorHojaDeVidaValida.java`
- `src/main/java/co/edu/cue/practicas/pattern/chain/ValidadorPracticaAnteriorFinalizada.java`

TEMPLATE METHOD — GPE-145

- `src/main/java/co/edu/cue/practicas/pattern/template/PlantillaValidacionAptitud.java`

PROXY — GPE-146 / GPE-147 / GPE-151

- `src/main/java/co/edu/cue/practicas/service/expediente/ExpedienteService.java` (Protection + Cache Proxy)

OBSERVER — GPE-143 / GPE-147 / GPE-150 / GPE-151

- `src/main/java/co/edu/cue/practicas/event/EmpresaObserver.java`
- `src/main/java/co/edu/cue/practicas/event/TutorInactivacionObserver.java`
- `src/main/java/co/edu/cue/practicas/event/CoordinadorNotificacionObserver.java`
- `src/main/java/co/edu/cue/practicas/event/listener/ExpedienteEventListener.java`

STATE — GPE-152 / GPE-153

- `src/main/java/co/edu/cue/practicas/model/entity/Vacante.java`
- `src/main/java/co/edu/cue/practicas/model/enums/EstadoVacante.java`

Sprint 3

1) OBSERVER (Patrón de Comportamiento)

Sí está implementado.

Archivos

- PracticasEmpresariales/backend/src/main/java/co/edu/cue/practicas/event/EmpresaObserver.java
- PracticasEmpresariales/backend/src/main/java/co/edu/cue/practicas/event/CoordinadorNotificacionObserver.java
- PracticasEmpresariales/backend/src/main/java/co/edu/cue/practicas/event/TutorInactivacionObserver.java
- PracticasEmpresariales/backend/src/main/java/co/edu/cue/practicas/service/Empresa/EmpresaService.java
- PracticasEmpresariales/backend/src/main/java/co/edu/cue/practicas/audit/singleton/AuditoriaLogger.java

Evidencia

• EmpresaObserver.java (línea 4): Define interfaz del observador con método onEmpresaEvento()
que notifica sobre eventos: "EMPRESA_APROBADA" | "EMPRESA_RECHAZADA" | "EMPRESA_INACTIVADA"

• CoordinadorNotificacionObserver.java (línea 7): Implementación concreta del Observer

que notifica al coordinador cuando ocurre un evento de empresa.

- TutorInactivacionObserver.java (línea 9): Implementación concreta que inactiva automáticamente

los tutores cuando la empresa se inactiva (regla GPE-151).

- EmpresaService.java (línea 61): Inyecta List<EmpresaObserver> y notifica a todos los observers cuando ocurren cambios de estado → SOLID DIP (Dependency Inversion Principle).

- AuditoriaLogger.java (línea 37): Utiliza ApplicationEventPublisher para publicar eventos de auditoría y que los listeners (observers) reaccionen (ej. alertas de seguridad).

2) SINGLETON (Patrón Creacional)

Sí está implementado.

Archivos

- PracticasEmpresariales/backend/src/main/java/co/edu/cue/practicas/audit/singleton/AuditoriaLogger.java
- PracticasEmpresariales/backend/src/main/java/co/edu/cue/practicas/config/singleton/SystemConfig.java
- PracticasEmpresariales/backend/src/main/java/co/edu/cue/practicas/service/empresa/EmpresaService.java

Evidencia

- AuditoriaLogger.java (línea 13): PATRÓN SINGLETON — GPE-139
Instancia única que recibe eventos de todos los módulos del sistema.
Centraliza el registro de acciones críticas garantizando trazabilidad completa.
La bitácora es INMUTABLE: no se puede modificar ni eliminar entradas creadas.
Regla de negocio: si falla el registro en bitácora, la operación se aborta.

- SystemConfig.java (línea 7): PATRÓN SINGLETON — GPE-136, GPE-140
Configuración global del sistema leída de application.properties.
Spring gestiona como bean singleton, garantizando única instancia en contexto de aplicación.
Centraliza parámetros: app.nombre, app.universidad, app.jwt.expiration-ms, app.mail.*

- EmpresaService.java (línea 42): PATRÓN SINGLETON

Spring gestiona este bean Service como singleton por defecto.

Anotación @Service + Bean management de Spring = Singleton implícito.

3) BUILDER (Patrón Creacional)

Sí está implementado.

Archivos

- PracticasEmpresariales/backend/src/main/java/co/edu/cue/practicas/pattern/builder/EmpresaBuilder.java
- PracticasEmpresariales/backend/src/main/java/co/edu/cue/practicas/service/empresa/EmpresaService.java
- PracticasEmpresariales/backend/src/main/java/co/edu/cue/practicas/model/entity/BitacoraAuditoria.java

Evidencia

- EmpresaBuilder.java (línea 11): PATRÓN BUILDER — ConcreteBuilder

Construye una Empresa paso a paso separando construcción de representación final.

Métodos: conRazonSocial(), conNit(), conSector(), conDireccion(), conTelefono(), conContacto(), conAreas(), build()

SOLID — SRP: solo construye Empresa. SOLID — OCP: nuevos campos = nuevos métodos.

- EmpresaService.java (línea 70): Uso de EmpresaBuilder en crearEmpresa()

Instancia builder y encadena métodos: new EmpresaBuilder()

.conRazonSocial("TechCo S.A.")

.conNit("900.123.456-7")

.conSector("Tecnología")

.conContacto("Juan", "juan@techco.com")

.build();

- BitacoraAuditoria.java: Builder patrón implícito via Lombok @Builder

Permite construcción fluida de entidades de auditoría.

4) PROTOTYPE (Patrón Creacional)

Sí está implementado.

Archivos

- PracticasEmpresariales/backend/src/main/java/co/edu/cue/practicas/pattern/p
rototype/EmpresaPlantilla.java
- PracticasEmpresariales/backend/src/main/java/co/edu/cue/practicas/pattern/p
rototype/IPrototype.java
- PracticasEmpresariales/backend/src/main/java/co/edu/cue/practicas/service/e
mpresa/EmpresaService.java

Evidencia

- EmpresaPlantilla.java (línea 11): PATRÓN PROTOTYPE — ConcretePrototype
Del diagrama: IPrototype → PracticeTemplate → (clona para crear) →

PracticeInstance

Cuando Coordinador registra empresa con estructura similar a existente, clona
plantilla

y solo ajusta NIT y razón social. Clon siempre inicia en PENDIENTE.

SOLID — SRP: solo clona configuración. SOLID — OCP: nuevos campos
clonables sin tocar.

- IPrototype.java: Interfaz genérica <T> define contrato clonar(): T

- EmpresaService.java (línea 95): Uso de EmpresaPlantilla en
crearDesdeEmpresaExistente()

Empresa clon = new EmpresaPlantilla(origen).clonar();

Clona sector, dirección, municipio, teléfono, contacto, áreas — no NIT ni razón
social.

5) PROXY (Patrón Estructural)

Sí está implementado.

Archivos

- PracticasEmpresariales/backend/src/main/java/co/edu/cue/practicas/security/f
ilter/ScopeValidationAspect.java
- PracticasEmpresariales/backend/src/main/java/co/edu/cue/practicas/service/e
xpediente/ExpedienteService.java

- PracticasEmpresariales/backend/src/main/java/co/edu/cue/practicas/model/entity/ExpedienteEstudiante.java
- PracticasEmpresariales/backend/src/main/java/co/edu/cue/practicas/service/estudiante/EstudianteService.java

Evidencia

- ScopeValidationAspect.java: Actúa como PROTECTION PROXY con @Aspect y @Around
Intercepta todos los métodos anotados con @RequiereRol, bloqueando acceso por rol.
Valida permiso ANTES de ejecutar: si falla, lanza 403 y registra en AuditoriaLogger.
- ExpedienteService.java: Usa CACHE/PROTECCIÓN PROXY para expedientes
Aplica filtrado por rol en backend, también descrito como proxy en comentarios.
- ExpedienteEstudiante.java: Documentado como utilizando reglas de "inmutabilidad/proxy"
Protege integridad de datos del expediente a través de validaciones.
- EstudianteService.java: Aplica filtrado por rol en backend, también descrito como proxy
Implementa control de acceso por rol a operaciones de estudiante.

6) STATE (Patrón Comportamiento)

Sí está implementado.

Archivos

- PracticasEmpresariales/backend/src/main/java/co/edu/cue/practicas/model/entity/InstanciaPractica.java
- PracticasEmpresariales/backend/src/main/java/co/edu/cue/practicas/model/entity/PlanPractica.java
- PracticasEmpresariales/backend/src/main/java/co/edu/cue/practicas/model/entity/SeguimientoSemanal.java
- PracticasEmpresariales/backend/src/main/java/co/edu/cue/practicas/model/enums/EstadoPractica.java
- PracticasEmpresariales/backend/src/main/java/co/edu/cue/practicas/model/enums/EstadoSeguimiento.java

Evidencia

- InstanciaPractica.java: Implementa STATE para transiciones de práctica
Estados: ASIGNADA_PENDIENTE_INICIO → EN_CURSO → FINALIZADA/CANCELADA
Métodos como confirmarVinculacion() validan estado actual antes de transicionar.
Encapsula validaciones de transición dentro de la entidad.
- PlanPractica.java: Implementa STATE para plan de práctica
Gestiona transiciones de estado con validaciones internas.
- SeguimientoSemanal.java: Implementa STATE para reporte de seguimiento
Estados: PENDIENTE → APROBADO/RECHAZADO
Métodos: aprobar(docenteld), rechazar(docenteld, motivoRechazo)
Valida que estado sea PENDIENTE antes de transicionar, lanza OperacionNoPermitidaException si no.
- EstadoPractica.java (enum): Define estados permitidos para InstanciaPractica
- EstadoSeguimiento.java (enum): Define estados permitidos para SeguimientoSemanal

7) FACADE (Patrón Estructural)

Sí está implementado.

Archivos

- PracticasEmpresariales/backend/src/main/java/co/edu/cue/practicas/service/vinculacion/VinculacionService.java
- PracticasEmpresariales/backend/src/main/java/co/edu/cue/practicas/service/dashboard/DashboardIndicadorService.java
- PracticasEmpresariales/backend/src/main/java/co/edu/cue/practicas/service/asignacion/AsignacionService.java

Evidencia

- VinculacionService.java: FACADE que orquesta confirmación de vinculación de práctica
Coordina múltiples operaciones: actualiza estado InstanciaPractica, persiste cambios,
publica evento VinculacionConfirmadaEvent, registra auditoría.

Método public InstanciaPracticaResponse confirmarVinculacion() actúa como punto único de entrada simplificando para el cliente.

- DashboardIndicadorService.java: FACADE para agregación de métricas de dashboard

Calcula indicadores role-específicos desde múltiples repositorios en un solo método.

public DashboardIndicadores obtenerIndicadores() retorna datos consolidados sin exponer complejidad de múltiples consultas.

- AsignacionService.java: FACADE para orquestar proceso de asignación

Agrupar operaciones de creación, actualización y validación de asignaciones.

Sprint 4

Template Method

- service/evaluacion/AbstractEvaluacionFinalTemplate.java
- service/evaluacion/EvaluacionDocenteService.java
- service/evaluacion/EvaluacionTutorService.java

State

- model/entity/EvaluacionFinal.java (método completar())
- model/entity/EncuestaSatisfaccion.java (métodos enviar(), guardarBorrador(), completar())
- model/entity/InstanciaPractica.java (método finalizarConResultado())

Observer

- event/Sprint4DomainEvent.java
- event/listener/Sprint4ObserverListener.java

Chain of Responsibility + Decorator

- service/cierre/ChecklistCierreService.java

Facade

- service/cierre/CierreFormalFacade.java

Proxy

- `model/entity/EvaluacionFinal.java` (bloqueo post-cierre en `completar()`)
- `model/entity/NotaFinalCoordinador.java` (bloqueo en `actualizar()`)

Strategy

- `service/evaluacion/NotaFinalCoordinadorService.java`
- `service/configuracion/ProgramaConfiguracionService.java`

Builder

- `service/reporte/ReporteEstadoProcesoService.java`
- `service/reporte/TableroGerencialDireccionService.java`

Adapter

- `service/notificacion/NotificacionConfigurableService.java`
- `service/evaluacion/EvaluacionTutorService.java`

Singleton

- `service/notificacion/NotificacionConfigurableService.java`

Mediator

- `service/cierre/CierreFormalFacade.java` (coordinación de notificaciones)

Ruta base analizada:

PracticasEmpresariales/backend/src/main/java/co/edu/cue/practicas/

Tabla principal de patrones y conexiones

Patrón	Clase / ruta principal	Clases relacionadas	Conexión con el resto del código	Cómo se implementa en el código
Singleton	audit/singleton/AuditoriaLogger.java	BitacoraAuditoriaRepository; ApplicationEventPublisher; event/AuditoriaEvent.java; event/listener/AuditoriaEventListener.java; security/filter/ScopeValidationAspect.java; servicios como AuthService, UsuarioService, ProgramaService, AsignacionService, VinculacionService.	AuditoriaLogger es un @Component de Spring, por lo tanto Spring lo administra como una única instancia. Muchos servicios lo inyectan para registrar acciones críticas. Después de guardar la bitácora publica AuditoriaEvent para que los listeners reaccionen.	Se implementa mediante singleton administrado por Spring. La clase tiene dependencias finales, recibe el repositorio de bitácora y el publicador de eventos por constructor, y ofrece registrar(...) como punto central para guardar auditoría. La conexión principal es: Servicio/Aspecto -> AuditoriaLogger -> BitacoraAuditoriaRepository -> AuditoriaEvent -> AuditoriaEventListener.
Proxy	security/filter/ScopeValidationAspect.java	security/annotation/RequiereRol.java; security/annotation/SoloLectura.java; security/CustomUserDetails.java; AuditoriaLogger.java; servicios anotados como EmpresaService, EstudianteService, ExpedienteService, CatalogoPracticaService.	El aspecto se interpone antes de ejecutar métodos anotados con @RequiereRol o @SoloLectura. Lee la anotación, obtiene el usuario autenticado desde SecurityContextHolder, valida el rol y solo deja pasar con joinPoint.proceed() si tiene permiso.	Es un Protection Proxy con Spring AOP: @Aspect + @Around. La conexión es: Controller llama Service -> el Aspect intercepta -> valida rol/scope -> si falla lanza AccesoNoAutorizadoException y registra auditoría -> si pasa ejecuta el método real.
Observer	event/EmpresaObserver.java + service/empresa/EmpresaService.java	event/CoordinadorNotificacionObserver.java; event/TutorInactivacionObserver.java; model/entity/Empresa.java; repository/tutor/TutorEmpresarialRepository.java.	EmpresaService depende de List<EmpresaObserver>, no de clases concretas. Spring inyecta automáticamente todos los @Component que implementan EmpresaObserver.	Se implementa con una interfaz observadora y observadores concretos. La conexión principal es: EmpresaService -> List<EmpresaObserver> -> Empresa.agregarOb

			Cuando se aprueba, rechaza o inactiva una empresa, EmpresaService registra los observers en la entidad y dispara el evento.	server(...) -> cambio de estado de Empresa -> onEmpresaEvento(...). CoordinadorNotificacionObserver notifica, TutorInactivacionObserver reacciona inactivando tutores.
Builder	pattern/builder/Empr esaBuilder.java	service/empresa/Em presaService.java; model/entity/Empr esa.java; dto/request/CrearEm presaRequest.java; model/enums/Estado Empresa.java.	EmpresaService usa EmpresaBuilder en crearEmpresa(...). El request trae los datos, el builder los va asignando con métodos encadenados y finalmente construye la entidad Empresa lista para guardar.	Se implementa con métodos fluidos: conRazonSocial(...), conNit(...), conSector(...), conDireccion(...), conTelefono(...), conContacto(...), conAreas(...), build(). La conexión es: CrearEmpresaReque st -> EmpresaService -> EmpresaBuilder -> Empresa.builder() -> EmpresaRepository. save(...).
Prototype	pattern/prototype/Em presaPlantilla.java	pattern/prototype/IPr ototype.java; service/empresa/Em presaService.java; model/entity/Empr esa.java; EmpresaRepository.	EmpresaService usa new EmpresaPlantilla(ori gen).clonar() cuando se quiere crear una empresa nueva tomando como base una empresa existente. Luego cambia los datos únicos como NIT y razón social antes de guardar.	Se implementa con IPrototype<T> y el método clonar(). EmpresaPlantilla copia sector, dirección, municipio, teléfono, contacto y áreas, pero no copia datos únicos. La conexión es: EmpresaRepository.f indByld(origen) -> EmpresaPlantilla.clo nar() -> ajustes de nuevoNit/nuevaRazo nSocial -> EmpresaRepository. save(clon).
Strategy	pattern/strategy/Estr ategiaValidacionEsta ndar.java	pattern/strategy/Estr ategiaValidacion.java ; pattern/template/Pla ntillaValidacionAptitu d.java; service/estudiante/E studienteService.jav a; validadores de pattern/chain.	EstudianteService no depende directamente de una validación concreta, sino de la interfaz EstrategiaValidacion. La estrategia estándar construye la cadena de validadores que debe ejecutarse para saber si el estudiante es apto.	Se implementa con una interfaz de estrategia y una clase concreta @Component. La conexión es: EstudianteService -> PlantillaValidacionAp titud -> EstrategiaValidacion. construirCadena() -> ValidadorAptitud.vali dar(ctx). Si luego hay reglas diferentes, se crea otra estrategia.

Chain of Responsibility	pattern/chain/ValidadorAptitud.java	ValidadorCatalogoActivo.java; ValidadorHojaDeVidaValida.java; ValidadorPracticaAnteriorFinalizada.java; ContextoValidacion.java; EstrategiaValidacionEstandar.java.	La estrategia estándar crea una cadena de validadores. Cada validador revisa una regla específica y si pasa delega al siguiente. Si falla, detiene el proceso lanzando excepción.	Se implementa con un handler abstracto que tiene setSiguiente(...) y validar(ctx). La conexión es: EstrategiaValidacionEstandar crea eslabón 1, 2 y 3 -> setSiguiente(...) -> PlantillaValidacionAptitud llama validar(ctx) -> cada validador ejecuta su regla.
Template Method	pattern/template/PlantillaValidacionAptitud.java	service/estudiante/EstudianteService.java; EstrategiaValidacion.java; ContextoValidacion.java; Usuario; CatalogoPractica; HojaDeVida; InstanciaPractica.	La plantilla define un flujo fijo para validar aptitud. EstudianteService extiende la plantilla y completa el hook onAptoConfirmado(..), donde puede actualizar estado, crear instancia o aplicar acciones del negocio.	Se implementa con un método final ejecutar(...). Ese método construye el contexto, ejecuta la estrategia/cadena y luego llama el hook protegido. La conexión es: Controller/servicio llama EstudianteService -> método heredado ejecutar(...) -> Strategy + Chain -> onAptoConfirmado(..) en EstudianteService.
State	model/entity/InstanciaPractica.java	model/enums/EstadoPractica.java; service/vinculacion/VinculacionService.java; service/cierre/CierreFormalFacade.java; service/asignacion/AsignacionService.java; repository/expediente/InstanciaPracticaRepository.java.	InstanciaPractica controla sus propios estados. Los servicios no cambian el estado libremente, sino que llaman métodos del dominio como iniciar(), finalizar(), cancelar(), confirmarVinculacion(...) o finalizarConResultado(...).	Se implementa con un enum de estados y métodos que validan el estado actual antes de transicionar. La conexión es: Servicio de negocio -> InstanciaPractica.métodoDeTransición() -> validarEstado(...) -> cambio de EstadoPractica -> repository.save(...).
Facade	service/cierre/CierreFormalFacade.java	controller/cierre/CierrePracticaController.java; ChecklistCierreService.java; NotificacionConfigurableService.java; InstanciaPracticaRepository; NotaFinalCoordinadorRepository; PazYSalvoRepository;	CierrePracticaController llama a una sola fachada para ejecutar el cierre formal. La fachada coordina validación de checklist, nota final, paz y salvo, cambio de estado, notificaciones y evento de dominio.	Se implementa como servicio orquestador. La conexión es: Controller -> CierreFormalFacade.ejecutar(...) -> ChecklistCierreService.generar(...) -> InstanciaPractica.finalizarConResultado(..) -> PazYSalvoRepository ->

		ApplicationEventPublisher.		NotificacionConfigurableService.enviar(...)-> Sprint4DomainEvent.
Mediator	service/dashboard/DashboardMediator.java	service/dashboard/DashboardService.java; controller/dashboard/DashboardController.java; service/dashboard/DashboardIndicadorService.java; dto/response/DashboardResponse.java; security/CustomUserDetails.java.	DashboardController no decide qué panel mostrar. DashboardService delega a DashboardMediator y el mediador decide qué estructura de DashboardResponse devolver según el rol del usuario.	Se implementa como componente coordinador. La conexión es: DashboardController -> DashboardService -> DashboardIndicadorService obtiene métricas -> DashboardMediator.resolveDashboard(user, indicadores) -> DashboardResponse por rol.
Decorator	service/cierre/ChecklistCierreService.java	CierreFormalFacade.java; EvaluacionFinalRepository; NotaFinalCoordinadorRepository; EncuestaSatisfaccionRepository; PracticaDocumentoRepository; SustentacionPracticaRepository; ProgramaConfiguracionService.	El checklist inicia como una estructura base y se va enriqueciendo con ítems según repositorios y configuración del programa. Cada requisito agrega información al resultado final sin cambiar el controlador.	Se implementa de forma conceptual: un Map base recibe varios ChecklistItemResponse. La conexión es: CierrePracticaController/CierreFormalFacade -> ChecklistCierreService.generar(...) -> consulta evaluación, nota, encuesta, documentos, sustentación y configuración -> retorna ChecklistCierreResponse decorado con todos los requisitos.
Adapter	service/notificacion/NotificacionConfigurableService.java	model/entity/PlantillaNotificacion.java; repository/notificacion/PlantillaNotificacionRepository.java; service/notificacion/EmailService.java; BitacoraCorreoRepository; CierreFormalFacade; EncuestaSatisfaccionService; ConfiguracionSprint4Controller.	El servicio adapta plantillas configurables del sistema al formato concreto que necesita EmailService para enviar correos. Toma asunto/cuerpo, reemplaza variables y luego invoca el envío real.	Se implementa adaptando datos de negocio a una llamada de infraestructura. La conexión es: módulo que necesita notificar -> NotificacionConfigurableService.enviar(tipo, actorId, destinatario, variables) -> PlantillaNotificacionRepository -> aplicarVariables(...) -> EmailService.notificarAsignacion(...) -> BitacoraCorreoRepository.

Template Method - Sprint 4	service/evaluacion/AbstractEvaluacionFinalTemplate.java	EvaluacionDocenteService.java; EvaluacionTutorService.java; EvaluacionFinalRepository; InstanciaPracticaRepository; EvaluacionFinal.java; Sprint4DomainEvent.	La clase abstracta define el flujo para registrar cualquier evaluación final. Las subclases solo cambian el tipo de evaluación, el evento publicado y la validación del evaluador.	Se implementa con registrar(...) como flujo fijo: buscar instancia editable -> validar evaluador -> crear/recuperar evaluación -> completar criterios -> guardar -> publicar evento. La conexión es: EvaluacionFinalController -> EvaluacionDocenteService/EvaluacionTutorService -> AbstractEvaluacionFinalTemplate.registrar(...) -> repositorios -> Sprint4DomainEvent.
-----------------------------------	---	--	--	---

Flujos resumidos para explicar en sustentación

Patrón	Flujo de conexión entre clases
Singleton	AuthService / UsuarioService / ScopeValidationAspect / otros servicios -> AuditoriaLogger -> BitacoraAuditoriaRepository -> AuditoriaEvent -> AuditoriaEventListener.
Proxy	Controller -> Service anotado con @RequiereRol -> ScopeValidationAspect intercepta -> valida SecurityContextHolder y CustomUserDetails -> deja pasar con joinPoint.proceed() o lanza 403.
Observer	EmpresaService -> List<EmpresaObserver> -> Empresa.agregarObserver(...) -> cambio de estado -> CoordinadorNotificacionObserver / TutorInactivacionObserver ejecutan reacción.
Builder	CrearEmpresaRequest -> EmpresaService.crearEmpresa(...) -> EmpresaBuilder.con...().build() -> EmpresaRepository.save(...).
Prototype	EmpresaRepository.findById(origen) -> new EmpresaPlantilla(origen).clonar() -> modificar NIT y razón social -> EmpresaRepository.save(clon).
Strategy + Chain	EstudianteService -> PlantillaValidacionAptitud -> EstrategiaValidacionEstandar.construirCadena() -> ValidadorCatalogoActivo -> ValidadorHojaDeVidaValida -> ValidadorPracticaAnteriorFinalizada.
State	VinculacionService / CierreFormalFacade / AsignacionService -> InstanciaPractica.iniciar/finalizar/cancelar/confirmarVinculacion -> validarEstado -> cambiar EstadoPractica.
Facade	CierrePracticaController -> CierreFormalFacade.ejecutar -> ChecklistCierreService -> InstanciaPractica -> PazYSalvoRepository ->

	NotificacionConfigurableService -> Sprint4DomainEvent.
Mediator	DashboardController -> DashboardService -> DashboardIndicadorService -> DashboardMediator -> DashboardResponse según rol.
Adapter	CierreFormalFacade o EncuestaSatisfaccionService -> NotificacionConfigurableService -> PlantillaNotificacionRepository -> aplicarVariables -> EmailService -> BitacoraCorreoRepository.